E-COMMERCE INTEGRATION

React Frontend -> Express Backend

A step-by-step developer blueprint outlining endpoints, credentials, local states, and client action gists for production deployment.

Target Audience:
Frontend Engineers, Full-Stack Architects & QA Teams

Generated: August 2026

Table of Contents

Guide Outline & Navigation

1. Setup & Environment Configurations

Bootstrap environment vars and Axios

Introduce dynamic configurations to prevent endpoints from leaking into visual layouts.

Step 1.1: Local Environment Variable Setup

Define values in your local browser-runtime environment context (.env) at the root level:

```
VITE_API_URL=https://api.yourdomain.com/api/v1
VITE_RAZORPAY_KEY_ID=rzp_test_productionKeys
```

Step 1.2: Centralized Axios Instance Service

Create src/lib/api.js to structure credentials forwarding. This tracks access cookies securely cross-origin:

```
import axios from 'axios';

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true, // Crucial for cross-origin cookie-session headers
});
```

2. Authentication & Session Services

Establishing user context & token lifetimes

Link login operations to REST endpoints. Keep users logged in by checking active credentials.

Step 2.1: App Provider Verification Check

On application startup in AppProvider.jsx, attempt a silent token checks using a background verify query:

```
const checkActiveSession = async () => {
  try {
    const response = await api.get('/auth/me');
    setSession(response.data.user);
  } catch {
    signOut(); // Handle expired context gracefully
  }
};
```

Step 2.2: Sign In Action mapping

Map login inputs in LoginPage.jsx to auth POST. On success, store the JWT in memory or localStorage:

```
const handleLogin = async (e) => {
  e.preventDefault();
  try {
    const res = await api.post('/auth/login', { email, password });
    login(res.data.user, res.data.accessToken);
    navigate('/admin');
  } catch (err) {
    toast.error("Invalid credentials.");
  }
};
```

Step 2.3: Password Recovery Link Requests

In ForgotPasswordPage.jsx, POST the email to recovery routes:

```
const handleForgotPassword = async (email) => {
  await api.post('/auth/forgot-password', { email });
  toast.success("Recovery instructions sent via email!");
};
```

3. Catalog Filtering & Detailed Views

Loading browse listings dynamically

Transition catalogue routes using query criteria to allow users to paginate, search, and category filter.

Step 3.1: Fetch List with Sorting Headers

Embed parameters inside ProductsPage.jsx using active sorting options:

```
const loadProducts = async () => {
  const params = {
    q: searchKeyword || undefined,
    category: activeCategory === 'All' ? undefined : activeCategory,
    sort: activeSort === 'Price low' ? 'price' : '-price'
  };
  const res = await api.get('/products', { params });
  setProductsList(res.data.data);
};
```

Step 3.2: Detailed Specs Map & Image Arrays

In ProductDetailPage.jsx, search products using slugs instead of raw Mongo IDs:

```
const fetchProductDetails = async () => {
  const res = await api.get(`/products/${slug}`);
  setProduct(res.data.data.product);
};
```

Design Detail: Image Render Guard

The database stores catalog pictures under an array of objects. Guard elements to load safely:

```
const imgUrl = product.images?.[0]?.url || product.images?.[0] || 'placeholder.png';
```

4. Shopping Cart API Mappings

Synchronizing offline items with database state

When logged out, store cart changes in local client state. Once authenticated, merge and push cart details to backing databases.

Step 4.1: Merge Local Cart Items on Login Session

Map localized carts inside AppProvider.jsx during user validation steps:

```
const mergeLocalCart = async (guestItems) => {
  for (const item of guestItems) {
    await api.post('/cart/items', {
      productId: item.product.id || item.product._id,
      quantity: item.quantity
    });
  }
  localStorage.removeItem('guestCart');
};
```

Step 4.2: Update and Remove Cart Services

Construct cart item mutations on CartPage.jsx to verify available inventory:

```
const modifyQuantity = async (productId, nextQty) => {
  await api.patch(`/cart/items/${productId}`, { quantity: nextQty });
  refreshCart(); // Pull DB update
};

const removeItem = async (productId) => {
  await api.delete(`/cart/items/${productId}`);
  refreshCart();
};
```

5. Checkout Orders & Idempotency Rules

Safe transactions via client-side keys

Prevent duplicate transactions or order creations caused by double-taps on weak networks.

Step 5.1: Idempotency Key Injection Header

Inside CheckoutPage.jsx, generate and assign a unique uuid parameter. Embed it inside custom request headers:

```
import { v4 as uuidv4 } from 'uuid';

const submitOrder = async (shippingDetails) => {
  const key = uuidv4();
  const res = await api.post('/checkout', { shippingDetails }, {
    headers: {
      'Idempotency-Key': key // Prevents duplicate checkouts on the backend
    }
  });
  return res.data.data.order;
};
```

Step 5.2: Cart Recovery Guard

Ensure that when an order registers successfully, you clear out cart buffers to ensure the checkout elements reset properly:

```
if (response.status === 201) {
  setCartItems([]);
  navigate(`/orders/${response.data.order.orderNumber}`);
}
```

6. Razorpay Payment Integrations

Initializing active payment checkout

Allow shoppers to process payments from the Order Details page for pending orders.

Step 6.1: Load Web payment script hooks

Create a script loader in OrderDetailPage.jsx:

```
const loadRazorpay = () => new Promise((resolve) => {
  const script = document.createElement('script');
  script.src = 'https://checkout.razorpay.com/v1/checkout.js';
  script.onload = () => resolve(true);
  script.onerror = () => resolve(false);
  document.body.appendChild(script);
});
```

Step 6.2: Launch Payment Options Window

Submit orders back-end to launch option frames:

```
const triggerPayment = async () => {
  const initialized = await loadRazorpay();
  if (!initialized) return toast.error("SDK load failed");

  const res = await api.post(`/payments/${orderNumber}`);
  const { razorpayOrder } = res.data.data;

  const options = {
    key: import.meta.env.VITE_RAZORPAY_KEY_ID,
    amount: razorpayOrder.amount,
    currency: razorpayOrder.currency,
    order_id: razorpayOrder.id,
    handler: (response) => {
      toast.success("Payment submitted!");
      refreshOrderDetails();
    }
  };
  new window.Razorpay(options).open();
};
```

7. Administrative Panel Operations

Backend retail operations & status overrides

Map admin tools to endpoints. Restrict access through auth gate controls.

Step 7.1: Catalog Product CRUD

Create, edit, or deactivate catalog items on AdminProductEditorPage.jsx:

```
// Create New Item
await api.post('/products', payload);

// Update Existing Item
await api.patch(`/products/${slug}`, payload);
```

Step 7.2: Inventory Stock Adjustments

Adjust stock levels on AdminInventoryPage.jsx using PATCH:

```
const adjustStock = async (productId, deltaCount) => {
  await api.patch(`/admin/products/${productId}/stock`, {
    delta: Number(deltaCount) // Increments or decrements stock count
  });
};
```

Step 7.3: Order Fulfillment Status Shifts

Fulfill order status states in AdminOrdersPage.jsx:

```
const updateStatus = async (orderId, newStatus) => {
  await api.patch(`/admin/orders/${orderId}/status`, {
    status: newStatus // processing, shipped, delivered, etc.
  });
};
```